

# Development Roadmap M1–M5

UM-CYDOS-007

| DOCUMENT     | REVISION         | STATUS  | CLASSIFICATION |
|--------------|------------------|---------|----------------|
| UM-CYDOS-007 | 1.0 · 2026-08-14 | current | Internal       |

## 1. Abstract

This report defines the remaining milestones to a working operating system, each with its technical content, principal risk, and exit criteria. Milestones are ordered so that each one's correctness can be established before the next depends on it.

## 2. Milestone summary

| ID | Deliverable                                       | Principal risk                     | State    |
|----|---------------------------------------------------|------------------------------------|----------|
| M0 | Kernel boots, self-checks pass                    | Link map errors                    | Complete |
| M1 | Timer interrupt and tick counter                  | Vector installation; silent faults | Next     |
| M2 | Two native tasks, preemptive switching            | Context save correctness           | —        |
| M3 | Heap allocator and VM memory model                | Fragmentation; arena sizing        | —        |
| M4 | Bytecode interpreter executing a program          | Instruction set design             | —        |
| M5 | Two VM applications time-sliced in bounded arenas | Isolation enforcement              | —        |

## 3. M1 — Timer interrupt and tick counter

**Deliverable.** A periodic timer interrupt increments a kernel tick counter; `kmain` observes it advancing. The spin-loop heartbeat from M0 is replaced by a timed one.

### Technical content

- Install an exception/interrupt vector table at the address the CPU expects ( `VECBASE` ).
- Write the interrupt entry and exit sequence. Under `call0` this is an explicit register save; the hardware does not do it.
- Configure a hardware timer and route it to a CPU interrupt level.
- Enable interrupts and confirm the handler runs without corrupting the interrupted context.
- Identify and either feed or disable any active watchdog.

**Principal risk.** A defective vector or entry sequence produces a reset with no output. The reset reason on the next boot is the only signal, and it is coarse. This is the milestone where the absence of JTAG hurts most.

**Prerequisite — resolve UM-CYDOS-004 §5.** M1 will push `.text` past the point where the kernel's IRAM segment overlaps the bootloader's working memory. This must be addressed before flashing, or M1 will fail during load with symptoms resembling a code defect.

**Exit criteria** 1. Tick counter advances at a measured, stable rate. 2. Interrupted code resumes correctly (verified by a checksum over registers across an interrupt). 3. System survives  $\geq 60$  seconds without reset.

## 4. M2 — Native task switching

**Deliverable.** Two kernel tasks alternate under a preemptive scheduler driven by the M1 timer, each maintaining private state across switches.

### Technical content

- Task control block: stack pointer, entry point, state.
- Per-task stack allocation with a guard pattern for overflow detection.
- Context switch in assembly: save `a0`, `a12 – a15` and live caller-saved registers, swap stack pointers, restore, return. **No window spilling** — see UM-CYDOS-003 §6.
- Round-robin scheduler invoked from the timer interrupt.
- A yield primitive for cooperative switching.

**Principal risk.** An incorrect register save corrupts state in a way that manifests later, in unrelated code. Classic symptom: a task runs correctly for several switches then faults in a function that did nothing wrong.

**Mitigation.** Have each task write a distinctive register pattern before yielding and verify it on resume. This converts silent corruption into an immediate, localised assertion failure.

**Exit criteria** 1. Two tasks alternate for  $\geq 10,000$  switches without corruption. 2. Register-pattern check passes on every resume. 3. Stack guard patterns intact.

## 5. M3 — Heap and VM memory model

**Deliverable.** A kernel allocator, and the arena model VM applications will run inside.

### Technical content

- Allocator over the DRAM region. A simple free-list is adequate; predictability matters more than throughput.
- Fixed-size **arenas** for VM applications — contiguous blocks with recorded base and length, the bounds the interpreter will enforce.
- Arena lifecycle: allocate on application start, release on exit.
- Instrumentation: high-water mark, fragmentation measure, allocation failure counter.

**Principal risk.** Arena sizing. Too small and applications cannot do useful work; too large and few can run concurrently. With  $\sim 176$  KB of DRAM total, minus kernel, stacks, and eventually a display buffer (UM-CYDOS-

004 §4 notes a full 16-bit framebuffer alone would be ~150 KB), the budget is genuinely tight and should be settled with measurements rather than guesses.

**Exit criteria** 1. Allocate/free cycles leave no leak across 10,000 iterations. 2. Arena bounds are queryable by the interpreter. 3. Out-of-memory returns a failure rather than corrupting.

---

## 6. M4 — Bytecode interpreter

---

**Deliverable.** The VM executes a trivial program — arithmetic, a loop, a syscall producing UART output.

### Technical content

- Instruction set design. Register-based is faster; stack-based is simpler to target. **Decision pending.**
- Dispatch loop with a per-instruction quantum counter — the mechanism enabling preemption at safe boundaries (UM-CYDOS-001 §4.2).
- **Bounds-checked memory access.** Every load and store validated against the arena. This is the isolation guarantee; it is not optional and must not be compiled out for performance.
- Syscall instruction trapping into kernel services.
- A producer: assembler or compiler for the bytecode.

**Principal risk.** Instruction set design is difficult to revise once a producer and programs exist. Keep it small — roughly 40 opcodes — and resist convenience additions.

**Secondary risk.** Interpretation overhead is unmeasured. If it proves unacceptable, the response is a threaded-dispatch or computed-goto interpreter, not abandoning bounds checking.

**Exit criteria** 1. Program computing a known result returns it correctly. 2. Out-of-bounds access is refused and reported, not performed. 3. Interpreter yields at quantum expiry. 4. Instructions-per-second measured and recorded.

---

## 7. M5 — Multiple applications

---

**Deliverable.** Two VM applications run concurrently, time-sliced, each confined to its own arena, with a misbehaving application unable to affect the other.

### Technical content

- Application table, scheduling, lifecycle.
- Per-application arena assignment and enforcement.
- Fault handling: an application violating its bounds is terminated, not the system.
- A minimal shell to start and stop applications.

**Principal risk.** This is where the isolation claim is tested. An application deliberately written to escape its arena must fail to do so.

**Exit criteria** 1. Two applications interleave correctly. 2. An application attempting out-of-bounds access is terminated; the other continues unaffected. 3. Terminating an application releases its arena completely.

---

## 8. Cross-cutting items

---

### 8.1 Version control — outstanding

Not yet initialised. The sibling project The Word Device was lost and recovered only because an editor's local history happened to retain it. This should be resolved before M1.

### 8.2 JTAG probe — ordered, not in hand

From M1 onward, failures are increasingly silent. UART cannot report a fault in the interrupt vector itself. Sequencing M1 after the probe arrives is recommended; if not, budget substantially more time.

### 8.3 Deferred decisions

| Decision                     | Needed by                | Notes                                        |
|------------------------------|--------------------------|----------------------------------------------|
| SMP or dedicated second core | M2                       | Fixes the scheduler's shape                  |
| Flash execute-in-place       | When kernel exceeds IRAM | Depends on L1 cache configuration            |
| Bytecode instruction set     | M4                       | Expensive to revise afterwards               |
| Filesystem strategy          | Post-M5                  | Writing FAT is a substantial subproject      |
| Display and touch drivers    | Post-M5                  | Framebuffer memory is the binding constraint |

### 8.4 Copy the borrowed binaries into this repository

`build.ps1` reads the bootloader and partition table from an unrelated project's build directory. Deleting or rebuilding that project breaks this one's flash step. Copying both into `cyd-os/boot/` removes the coupling — about 21 KB.

## 9. References

---

- UM-CYDOS-001 — architecture and the isolation model driving M4/M5
- UM-CYDOS-003 §6 — why M2 is tractable
- UM-CYDOS-004 §5 — the risk blocking M1
- UM-CYDOS-006 — what M0 established